



Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатика и управление

КАФЕДРА ИУК4 Программное обеспечение ЭВМ, информационные технологии

ЛАБОРАТОРНАЯ РАБОТА №2

«Основные примитивы OpenGL»

по дисциплине: *«Компьютерная графика»*

Выполнил: студент группы ИУК4-43Б

(Подпись)

Афанасьева Ю.Р.

(И.О. Фамилия)

Проверил:

(Подпись)

Тихонович Е.О.

(И.О. Фамилия)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

Цель: формирование практических навыков по работе с графическими примитивами OpenGL, а также применения к ним эффектов средствами машины состояний и использования проверки глубины.

Задачи:

научиться устанавливать размеры наблюдаемого объема, изучить параметры функции `glVertex`, изучить основные параметры функции `glBegin`, сформировать понимание особенности использования функции `glEnable` с конкретными геометрическими примитивами, выяснить основы построения сплошных объектов.

Вариант 3

Задание выполняется согласно варианту. По завершении готовится отчет.

- 1) Для каждого приложения установить размеры наблюдаемого объема.
- 2) Для [Листинга 1](#) задать с использованием тригонометрических преобразований (функций `sin` и `cos`) произвольное множество точек (не менее 30 точек) в трехмерном пространстве. Изучить принципы вращения экрана с помощью клавиш курсора и способы подключения меню к программе.
- 3) Для [Листинга 2](#) задать произвольное множество точек разного размера и цвета в трехмерном пространстве.
- 4) Для [Листинга 3](#) изобразить не менее 10 линий с разными параметрами (цвет, толщина, начертание по шаблону) в трехмерном пространстве.
- 5) Для [Листинга 4](#) выполнить аппроксимацию произвольной кривой линии посредством коротких прямых. На 3 примерах наглядно продемонстрировать точность аппроксимации. По возможности использовать меню.
- 6) Для [Листинга 5](#), используя `GL_LINE_STRIP` и `GL_LINE_LOOP` нарисовать трехмерный объект из ломанных линий содержащих не менее 15 точек. Точки задавать, используя цикл.
- 7) Для [Листинга 6](#) используя `GL_TRIANGLES` вывести на экран произвольную трехмерную геометрическую фигуру состоя-

шую из треугольников. Понимать принципы обхода точек треугольника и полигонов и знать на что он влияет.

- 8) Для [Листинга 7](#) продемонстрировать работу директив [GL_TRIANGLE_STRIP](#) и [GL_TRIANGLE_FAN](#).
- 9) Для [Листинга 8](#) для произвольно заданной фигуры составленной из не менее чем 6 треугольников продемонстрировать работу функции `glShadeModel`.
- 10) Для [Листинга 9](#) на наглядном примере продемонстрировать работу проверки глубины для технологии удаления скрытых поверхностей. На наглядном примере продемонстрировать работу технологии отбора задних граней.

Этапы выполнения работы:

1. Изучены основные графические примитивы OpenGL, правила задания вершин и требования к выполнению лабораторной работы.
2. Для каждого приложения установлен наблюдаемый объём и настроено сохранение пропорций объектов при изменении размеров окна.
3. Реализовано построение точек в трёхмерном пространстве с различными координатами, размерами и цветами.
4. Исследовано построение отдельных линий и ломаных, изменение их ширины и шаблона, а также аппроксимация кривой прямыми отрезками.
5. С помощью `GL_LINE_STRIP` и `GL_LINE_LOOP` построен трёхмерный объект, вершины которого вычисляются в циклах.
6. Реализованы трёхмерные фигуры с применением `GL_TRIANGLES`, `GL_TRIANGLE_STRIP` и `GL_TRIANGLE_FAN` и рассмотрен порядок обхода вершин.
7. Исследована работа плоского и плавного затенения, проверки глубины и отбора задних граней.
8. Выполнена проверка работы программ при вращении объектов и изменении размеров окна, получены изображения результатов

Задание:

- 2) Для [Листинга 1](#) задать с использованием тригонометрических преобразований (функций $\sin$ и $\cos$) произвольное множество точек (не менее 30 точек) в трехмерном пространстве. Изучить принципы вращения экрана с помощью клавиш курсора и способы подключения меню к программе.

Листинг 1

```
#include "glut.h"
#include <math.h>
#define GL_PI 3.1415f

static GLfloat xRot = 0.0f;
static GLfloat yRot = 0.0f;

void RenderScene(void)
{
    GLfloat x, y, z, angle;
    glClear(GL_COLOR_BUFFER_BIT);
    // сохраняем матрицу, чтобы поворот не накапливался при каждой перерисовке
    glPushMatrix();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);
    // sin и cos получают радианы; четыре оборота с шагом 0,1 дают около 252
    точек
    glBegin(GL_POINTS);

    z = -50.0f;
    for (angle = 0.0f; angle <= (2.0f * GL_PI) * 4.0f; angle += 0.1f)
    {
        x = 50.0f * sin(angle);
        y = 50.0f * cos(angle);
        glVertex3f(x, y, z);
        z += 0.4f;
    }
    glEnd();
    glPopMatrix();
    glutSwapBuffers();
}

void SetupRC(void)
{
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glColor3f(0.0f, 1.0f, 0.0f);
}

void SpecialKeys(int key, int x, int y)
{

```

```

    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;
    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;
    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;
    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    // после полного оборота возвращаем угол в диапазон от 0 до 355 градусов
    if (xRot > 355.0f)
        xRot = 0.0f;
    if (xRot < 0.0f)
        xRot = 355.0f;
    if (yRot > 355.0f)
        yRot = 0.0f;
    if (yRot < 0.0f)
        yRot = 355.0f;

    glutPostRedisplay();
}

void ProcessMenu(int value)
{
    if (value == 1)
    {
        xRot = 0.0f;
        yRot = 0.0f;
        glutPostRedisplay();
    }
}

void ChangeSize(int w, int h)
{
    GLfloat nRange = 100.0f;

    if (h == 0)
        h = 1;

    glViewport(0, 0, w, h);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // расширяем более длинную ось объема, чтобы фигура не растягивалась вместе с
    // окном
    if (w <= h)
        glOrtho(-nRange, nRange, -nRange * h / w, nRange * h / w,
                -nRange, nRange);
    else
        glOrtho(-nRange * w / h, nRange * w / h, -nRange, nRange,

```

```

        -nRange, nRange);

glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
}

int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutCreateWindow("Points Example");
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);
    SetupRC();

    glutCreateMenu(ProcessMenu);
    glutAddMenuEntry("Reset rotation", 1);
    glutAttachMenu(GLUT_RIGHT_BUTTON);

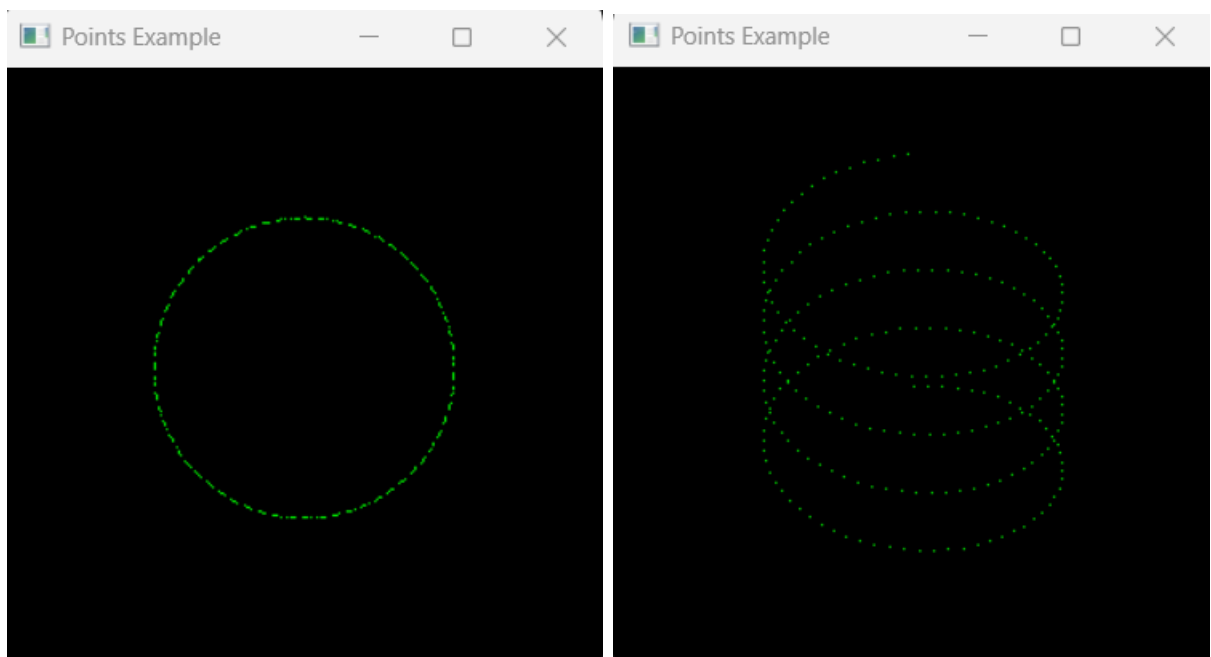
    glutMainLoop();

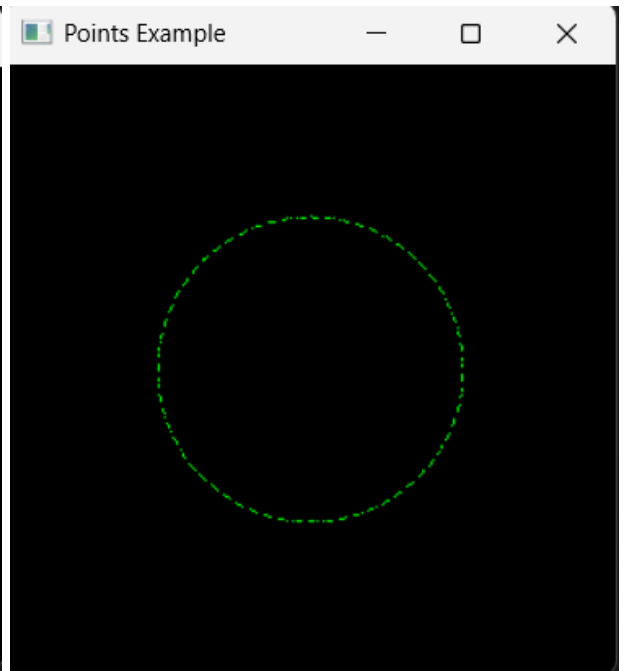
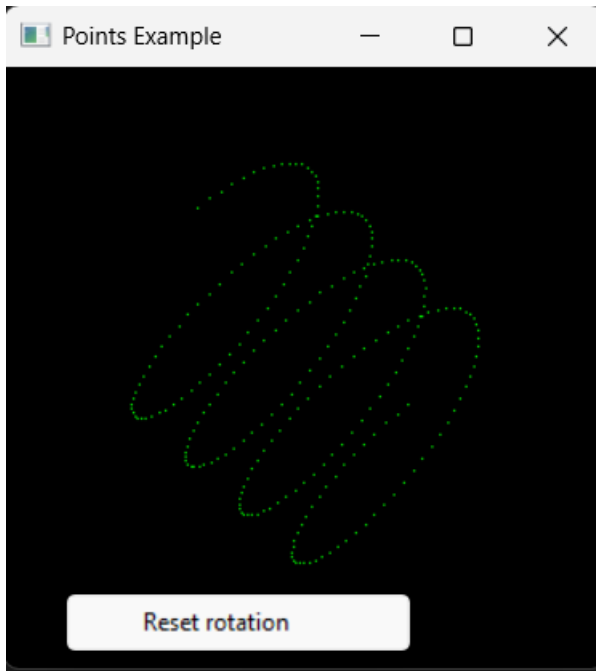
    return 0;
}

```

В программе реализована точечная трёхмерная спираль, координаты которой вычисляются функциями $\sin$ и $\cos$. Для отображения точек использован примитив `GL_POINTS`, вращение выполняется клавишами управления курсором, а контекстное меню возвращает объект к исходным углам поворота.

Результаты работы программы 1





Задание:

- 3) Для [Листинга 2](#) задать произвольное множество точек разного размера и цвета в трехмерном пространстве.

Листинг 2

```
#include "glut.h"

static GLfloat xRot = 0.0f;
static GLfloat yRot = 0.0f;

void RenderScene(void)
{
    glClear(GL_COLOR_BUFFER_BIT);

    // сохранение матрицы не позволяет повороту накапливаться при повторной
    // отрисовке
    glPushMatrix();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);

    glPointSize(2.0f);
    glColor3f(1.0f, 0.0f, 0.0f);
    glBegin(GL_POINTS);
    glVertex3f(-75.0f, -50.0f, -50.0f);
    glEnd();

    // выбранные размер и цвет действуют на следующие точки до очередного
    // изменения состояния
    glPointSize(4.0f);
    glColor3f(0.0f, 1.0f, 0.0f);
    glBegin(GL_POINTS);
    glVertex3f(-45.0f, 30.0f, -30.0f);
    glEnd();

    glPointSize(6.0f);
    glColor3f(0.0f, 0.0f, 1.0f);
    glBegin(GL_POINTS);
    glVertex3f(-15.0f, -20.0f, -10.0f);
    glEnd();

    glPointSize(8.0f);
    glColor3f(1.0f, 1.0f, 0.0f);
    glBegin(GL_POINTS);
    glVertex3f(15.0f, 50.0f, 10.0f);
    glEnd();

    glPointSize(9.0f);
    glColor3f(0.0f, 1.0f, 1.0f);
    glBegin(GL_POINTS);
```



```

    glVertex3f(45.0f, -40.0f, 30.0f);
    glEnd();

    // разные значения z задают точки в трехмерном пространстве, хотя проекция
    // остается ортографической
    glPointSize(10.0f);
    glColor3f(1.0f, 0.0f, 1.0f);
    glBegin(GL_POINTS);
    glVertex3f(75.0f, 20.0f, 50.0f);
    glEnd();

    glPopMatrix();
    glutSwapBuffers();
}

void SetupRC(void)
{
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;
    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;
    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;
    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    if (xRot > 355.0f)
        xRot = 0.0f;
    if (xRot < 0.0f)
        xRot = 355.0f;
    if (yRot > 355.0f)
        yRot = 0.0f;
    if (yRot < 0.0f)
        yRot = 355.0f;

    glutPostRedisplay();
}

void ChangeSize(int w, int h)
{
    GLfloat nRange = 100.0f;

    // нулевая высота заменяется единицей, чтобы исключить деление на ноль
    if (h == 0)
        h = 1;

```

```

glViewport(0, 0, w, h);
glMatrixMode(GL_PROJECTION);
glLoadIdentity();

// более длинная ось объема расширяется для сохранения пропорций при
изменении окна
if (w <= h)
    glOrtho(-nRange, nRange, -nRange * h / w, nRange * h / w,
            -nRange, nRange);
else
    glOrtho(-nRange * w / h, nRange * w / h, -nRange, nRange,
            -nRange, nRange);

glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
}

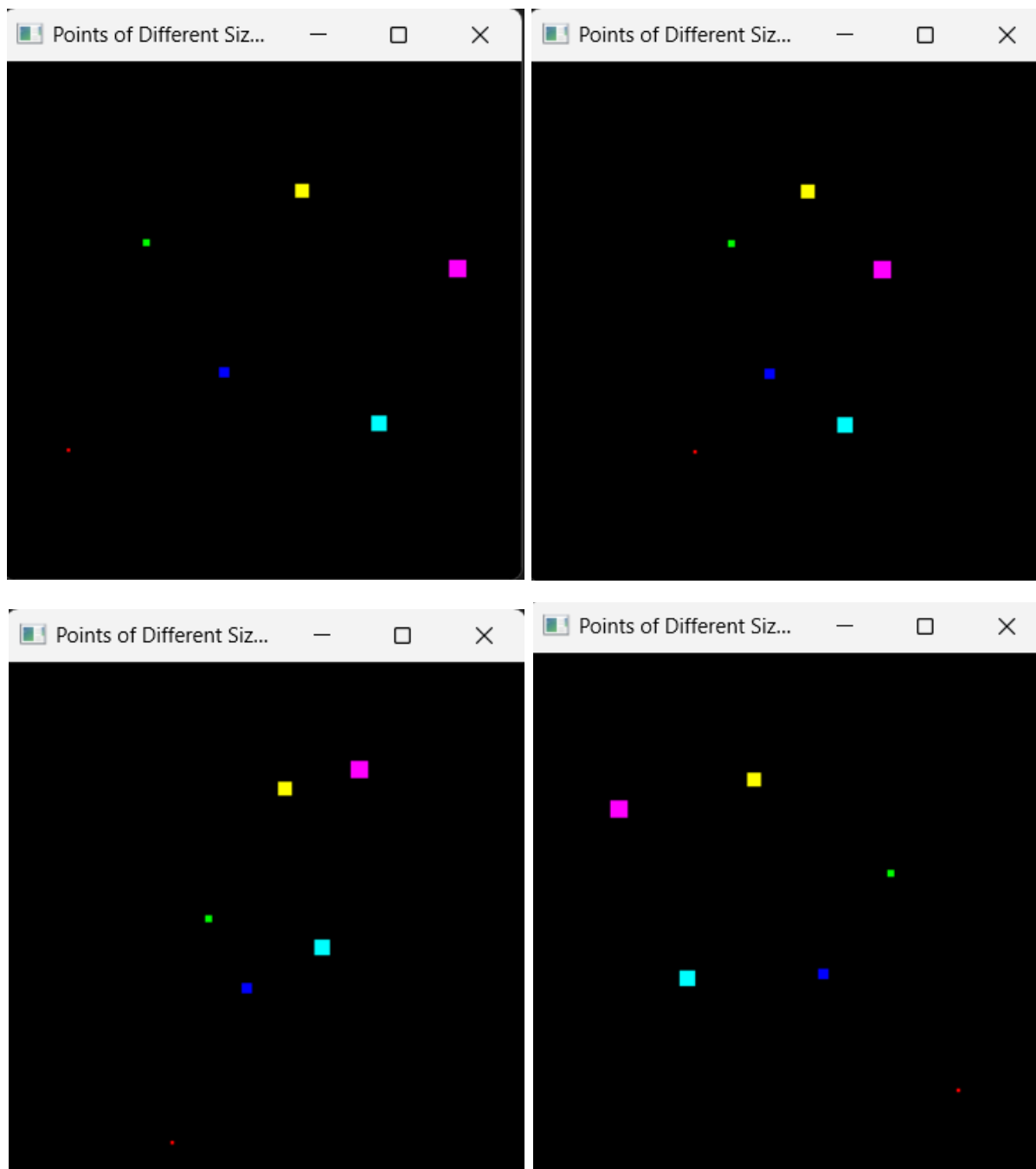
int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutCreateWindow("Points of Different Sizes and Colors");
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);
    SetupRC();
    glutMainLoop();

    return 0;
}

```

В программе построено множество точек с различными координатами x, y и z. Размер точек задаётся функцией `glPointSize`, цвет — функцией `glColor3f`. Вращение сцены позволяет наглядно увидеть различие координат по глубине.

Результаты работы программы 2



Задание:

- 4) Для [Листинга 3](#) изобразить не менее 10 линий с разными параметрами (цвет, толщина, начертание по шаблону) в трехмерном пространстве.

Листинг 3

```
#include "glut.h"

static GLfloat xRot = 0.0f;
static GLfloat yRot = 0.0f;

void RenderScene(void)
{
    const GLfloat colors[10][3] = {
        {1.0f, 0.0f, 0.0f},
        {0.0f, 1.0f, 0.0f},
        {0.0f, 0.0f, 1.0f},
        {1.0f, 1.0f, 0.0f},
        {0.0f, 1.0f, 1.0f},
        {1.0f, 0.0f, 1.0f},
        {1.0f, 0.5f, 0.0f},
        {0.5f, 1.0f, 0.0f},
        {0.0f, 0.5f, 1.0f},
        {1.0f, 1.0f, 1.0f}
    };
    const GLushort patterns[10] = {
        0xFFFF, 0x5555, 0x00FF, 0x0F0F, 0x3333,
        0x3F07, 0xAAAA, 0xF0F0, 0xFF00, 0x1111
    };
    glClear(GL_COLOR_BUFFER_BIT);

    // одинаковые индексы связывают цвет и шаблон с одной из десяти линий
    glPushMatrix();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);

    glEnable(GL_LINE_STIPPLE);

    for (int i = 0; i < 10; ++i)
    {
        GLfloat y = -90.0f + i * 20.0f;
        GLfloat z = -45.0f + i * 10.0f;

        // ширина и фактура задаются до glBegin, потому что внутри пары
        // glBegin/glEnd это недопустимо
        glLineWidth(1.0f + i);
        glColor3f(colors[i][0], colors[i][1], colors[i][2]);
```

```

        // единичный бит шаблона рисует участок линии, а нулевой оставляет
пропуск
        glLineStipple(1, patterns[i]);
        glBegin(GL_LINES);
        glVertex3f(-80.0f, y, z);
        glVertex3f(80.0f, y, z);
        glEnd();
    }

    // восстановление матрицы не позволяет повороту накапливаться при перерисовке
    glPopMatrix();
    glutSwapBuffers();
}

void SetupRC(void)
{
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;
    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;
    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;
    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;
    if (xRot > 355.0f)
        xRot = 0.0f;
    if (xRot < 0.0f)
        xRot = 355.0f;
    if (yRot > 355.0f)
        yRot = 0.0f;
    if (yRot < 0.0f)
        yRot = 355.0f;

    glutPostRedisplay();
}

void ChangeSize(int w, int h)
{
    GLfloat nRange = 100.0f;

    if (h == 0)
        h = 1;
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

```

```

        // более длинная ось объема расширяется, поэтому изменение окна не искажает
        // линии
        if (w <= h)
            glOrtho(-nRange, nRange, -nRange * h / w, nRange * h / w,
                    -nRange, nRange);
        else
            glOrtho(-nRange * w / h, nRange * w / h, -nRange, nRange,
                    -nRange, nRange);
        glMatrixMode(GL_MODELVIEW);
        glLoadIdentity();
    }

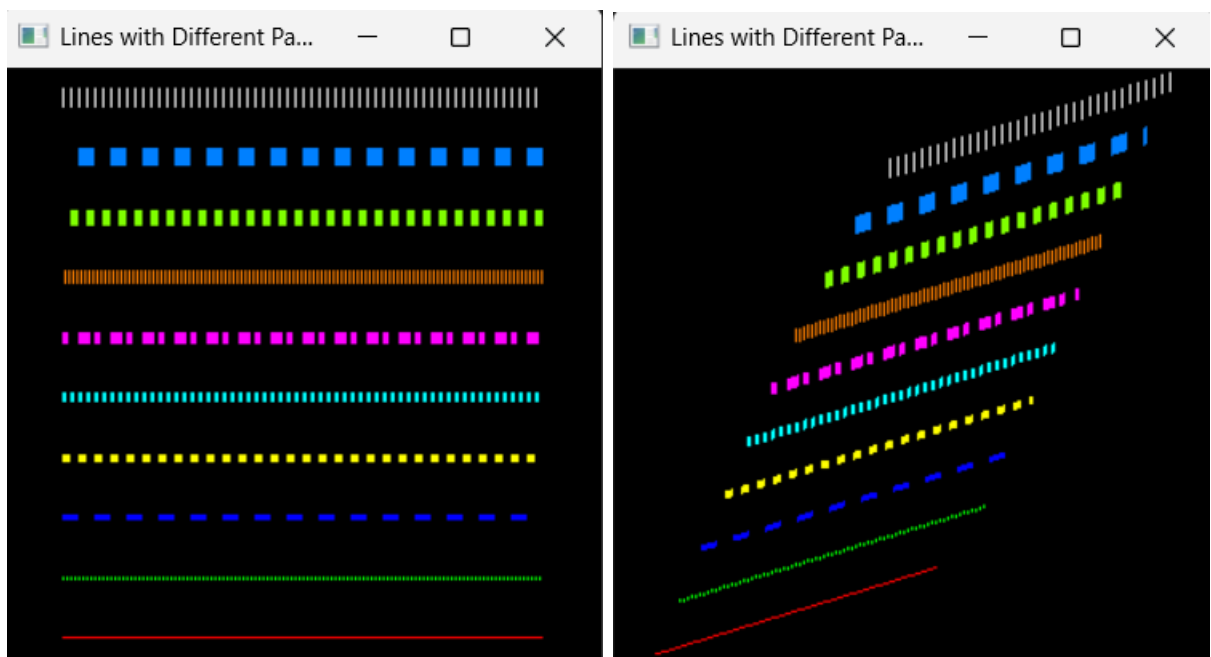
    int main(int argc, char* argv[])
    {
        glutInit(&argc, argv);
        glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
        glutCreateWindow("Lines with Different Parameters");
        glutReshapeFunc(ChangeSize);
        glutSpecialFunc(SpecialKeys);
        glutDisplayFunc(RenderScene);
        SetupRC();
        glutMainLoop();

        return 0;
    }

```

В программе построено десять линий в трёхмерном пространстве. Для линий заданы различные цвета, значения толщины и 16-битные шаблоны с применением `glLineWidth`, `glLineStipple` и `GL_LINE_STIPPLE`.

Результаты работы программы 3



Задание:

- 5) Для [Листинга 4](#) выполнить аппроксимацию произвольной кривой линии посредством коротких прямых. На 3 примерах наглядно продемонстрировать точность аппроксимации. По возможности использовать меню.

Листинг 4

```
#include "glut.h"
#include <math.h>

#define GL_PI 3.1415f

static int segmentCount = 8;

void RenderScene(void)
{
    GLfloat angle;

    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0f, 1.0f, 0.0f);

    // меньшее угловое расстояние между вершинами дает более короткие отрезки и
    // более точную кривую
    glBegin(GL_LINE_STRIP);
    for (int i = 0; i <= segmentCount; ++i)
    {
        angle = 2.0f * GL_PI * i / segmentCount;
        glVertex3f(70.0f * cos(angle), 70.0f * sin(angle), 0.0f);
    }
    glEnd();

    glutSwapBuffers();
}

void SetupRC(void)
{
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
}

void ProcessMenu(int value)
{
    // значение пункта меню сразу задает число отрезков в одном из трех примеров
    segmentCount = value;
    glutPostRedisplay();
}

void ChangeSize(int w, int h)
{
    GLfloat nRange = 100.0f;
```

```

    // замена нулевой высоты исключает деление на ноль при изменении размеров
окна
    if (h == 0)
        h = 1;

    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // расширение более длинной оси наблюдаемого объема сохраняет форму
окружности
    if (w <= h)
        glOrtho(-nRange, nRange, -nRange * h / w, nRange * h / w,
                -nRange, nRange);
    else
        glOrtho(-nRange * w / h, nRange * w / h, -nRange, nRange,
                -nRange, nRange);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutCreateWindow("Curve Approximation");
    glutReshapeFunc(ChangeSize);
    glutDisplayFunc(RenderScene);
    SetupRC();

    glutCreateMenu(ProcessMenu);
    glutAddMenuEntry("8 segments", 8);
    glutAddMenuEntry("24 segments", 24);
    glutAddMenuEntry("72 segments", 72);
    glutAttachMenu(GLUT_RIGHT_BUTTON);

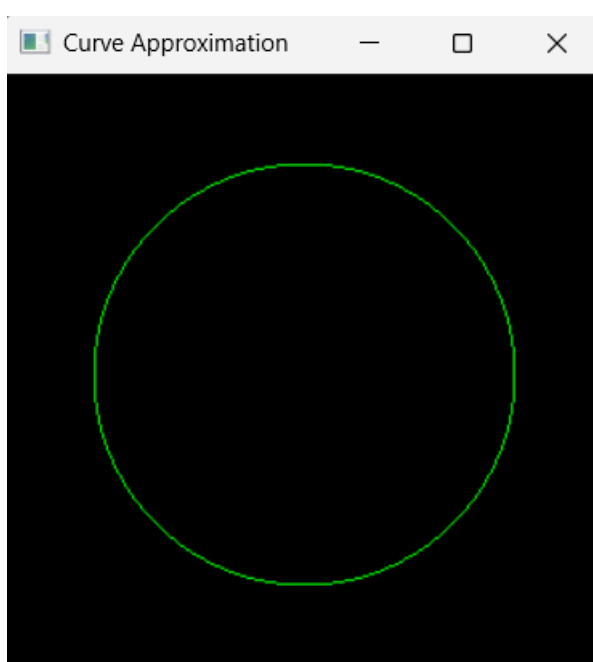
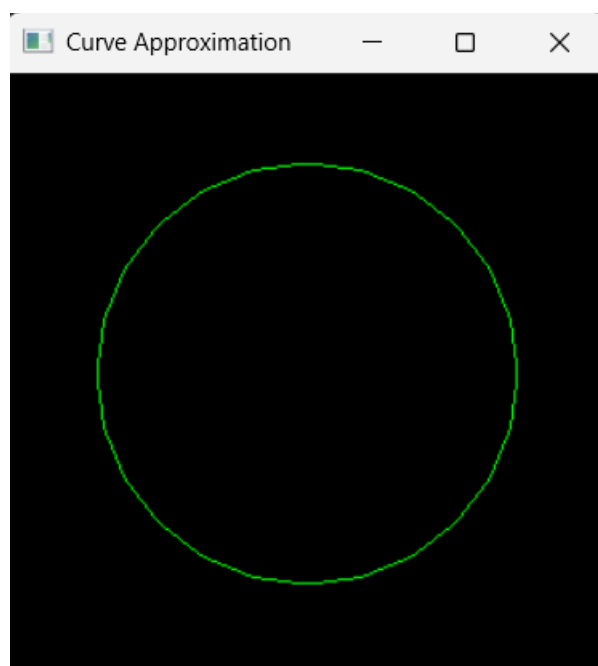
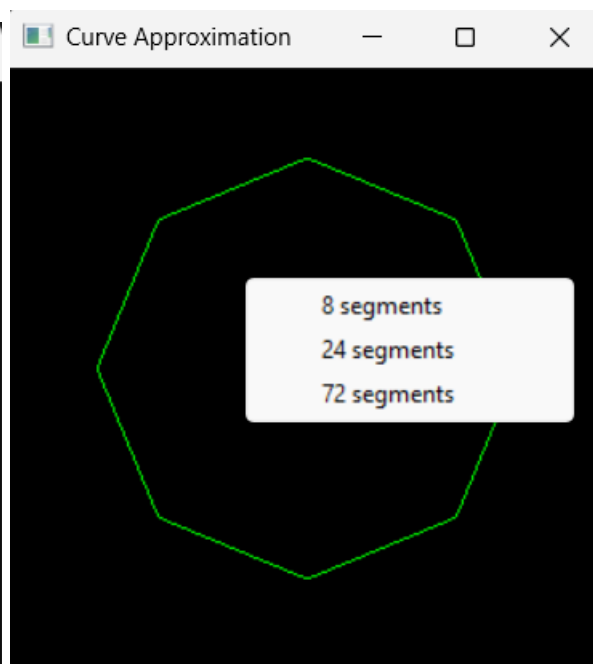
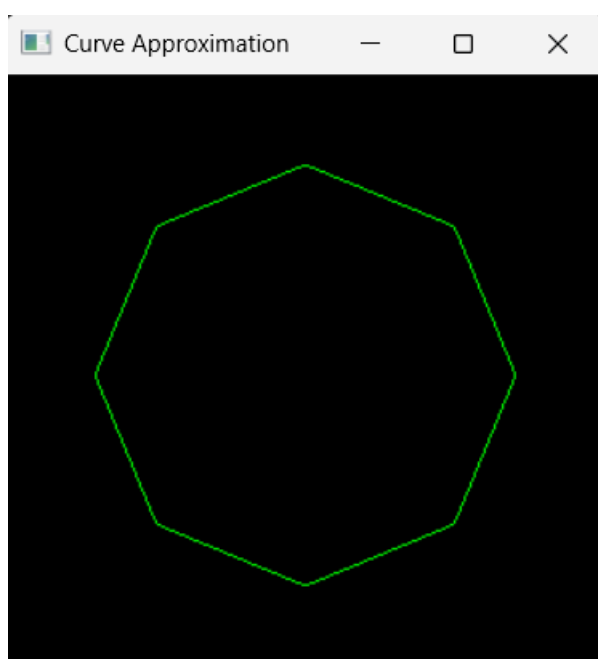
    glutMainLoop();

    return 0;
}

```

В программе выполнена аппроксимация окружности последовательностью прямых отрезков с помощью `GL_LINE_STRIP`. Через контекстное меню выбираются три уровня точности: 8, 24 и 72 отрезка. При увеличении числа отрезков контур становится более гладким.

Результаты работы программы 4



Задание:

- 6) Для [Листинга 5](#), используя `GL_LINE_STRIP` и `GL_LINE_LOOP` нарисовать трехмерный объект из ломанных линий содержащих не менее 15 точек. Точки задавать, используя цикл.

Листинг 5

```
#include "glut.h"
#include <math.h>
#define GL_PI 3.1415f

static GLfloat xRot = 0.0f;
static GLfloat yRot = 0.0f;

void RenderScene(void)
{
    const int pointCount = 20;
    GLfloat angle;

    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0f, 1.0f, 0.0f);

    // сохранение матрицы не позволяет повороту накапливаться при повторной
    отрисовке
    glPushMatrix();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);

    // GL_LINE_LOOP сам соединяет последнюю вершину с первой, поэтому повторять
    первую точку не нужно
    glBegin(GL_LINE_LOOP);
    for (int i = 0; i < pointCount; ++i)
    {
        angle = 2.0f * GL_PI * i / pointCount;
        glVertex3f(50.0f * cos(angle), 50.0f * sin(angle), -60.0f);
    }
    glEnd();
    glBegin(GL_LINE_LOOP);
    for (int i = 0; i < pointCount; ++i)
    {
        angle = 2.0f * GL_PI * i / pointCount;
        glVertex3f(50.0f * cos(angle), 50.0f * sin(angle), 60.0f);
    }
    glEnd();

    // GL_LINE_STRIP последовательно соединяет вершины, но не замыкает ломаную
    автоматически
    glBegin(GL_LINE_STRIP);
```

```

    for (int i = 0; i <= pointCount * 2; ++i)
    {
        angle = 4.0f * GL_PI * i / (pointCount * 2);
        glVertex3f(50.0f * cos(angle), 50.0f * sin(angle),
                  -60.0f + 120.0f * i / (pointCount * 2));
    }
    // две окружности содержат по 20 точек, а соединяющая их спираль – 41 точку
    glEnd();

    glPopMatrix();
    glutSwapBuffers();
}

void SetupRC(void)
{
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;
    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;
    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;
    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;
    if (xRot > 355.0f)
        xRot = 0.0f;
    if (xRot < 0.0f)
        xRot = 355.0f;
    if (yRot > 355.0f)
        yRot = 0.0f;
    if (yRot < 0.0f)
        yRot = 355.0f;

    glutPostRedisplay();
}

void ChangeSize(int w, int h)
{
    GLfloat nRange = 100.0f;

    if (h == 0)
        h = 1;
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    // расширение более длинной оси наблюдаемого объема сохраняет пропорции
    объекта

```

```

    if (w <= h)
        glOrtho(-nRange, nRange, -nRange * h / w, nRange * h / w,
                -nRange, nRange);
    else
        glOrtho(-nRange * w / h, nRange * w / h, -nRange, nRange,
                -nRange, nRange);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

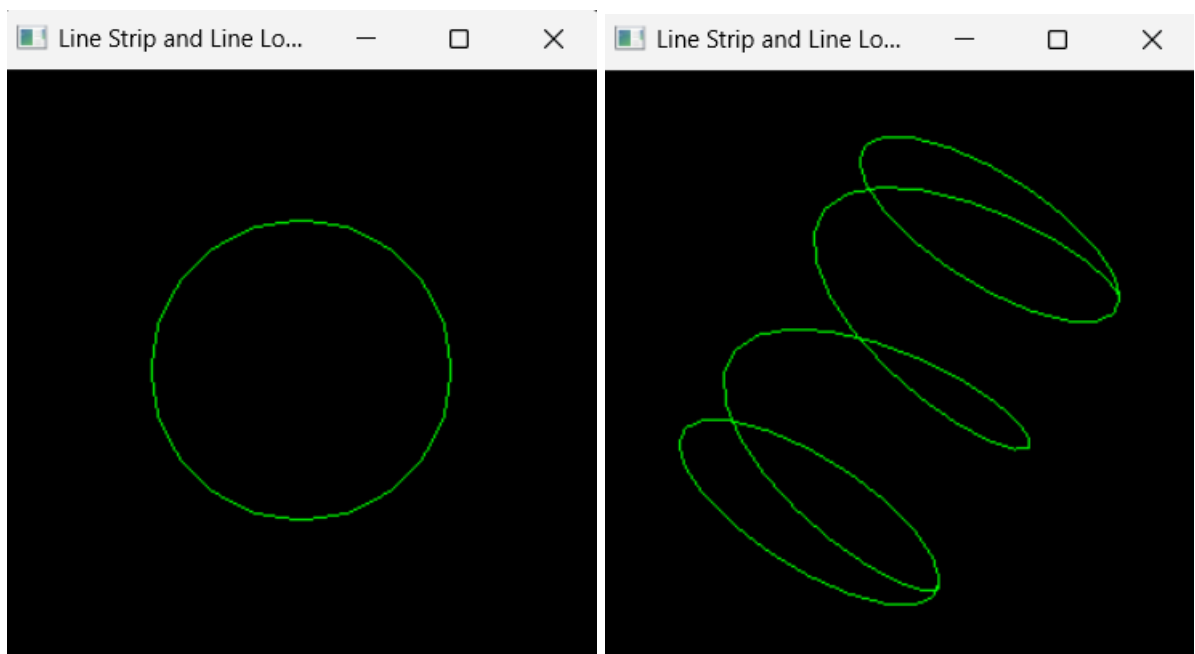
int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutCreateWindow("Line Strip and Line Loop Object");
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);
    SetupRC();
    glutMainLoop();

    return 0;
}

```

В программе построен трёхмерный проволочный цилиндрический объект. Два замкнутых контура созданы через `GL_LINE_LOOP`, а соединяющая их спираль — через `GL_LINE_STRIP`. Все вершины вычисляются в циклах, количество точек превышает требуемое значение.

Результаты работы программы 5



Задание:

- 7) Для [Листинга 6](#) используя GL_TRIANGLES вывести на экран произвольную трехмерную геометрическую фигуру состоящую из треугольников. Понимать принципы обхода точек треугольника и полигонов и знать на что он влияет.

Листинг 6

```
#include "glut.h"

static GLfloat xRot = 0.0f;
static GLfloat yRot = 0.0f;

void RenderScene(void)
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glPushMatrix();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);

    // каждая последовательная тройка вершин GL_TRIANGLES образует независимую
    // грань
    glBegin(GL_TRIANGLES);

    // четыре верхние грани обходятся против часовой стрелки при наблюдении
    // снаружи
    glColor3f(1.0f, 0.0f, 0.0f);
    glVertex3f(0.0f, 0.0f, 60.0f);
    glVertex3f(60.0f, 0.0f, 0.0f);
    glVertex3f(0.0f, 60.0f, 0.0f);

    glColor3f(0.0f, 1.0f, 0.0f);
    glVertex3f(0.0f, 0.0f, 60.0f);
    glVertex3f(0.0f, 60.0f, 0.0f);
    glVertex3f(-60.0f, 0.0f, 0.0f);

    glColor3f(0.0f, 0.0f, 1.0f);
    glVertex3f(0.0f, 0.0f, 60.0f);
    glVertex3f(-60.0f, 0.0f, 0.0f);
    glVertex3f(0.0f, -60.0f, 0.0f);

    glColor3f(1.0f, 1.0f, 0.0f);
    glVertex3f(0.0f, 0.0f, 60.0f);
    glVertex3f(0.0f, -60.0f, 0.0f);
    glVertex3f(60.0f, 0.0f, 0.0f);

    // у нижних граней порядок экваториальных вершин меняется для сохранения
    // внешнего обхода
    glColor3f(0.0f, 1.0f, 1.0f);
```

```

    glVertex3f(0.0f, 0.0f, -60.0f);
    glVertex3f(0.0f, 60.0f, 0.0f);
    glVertex3f(60.0f, 0.0f, 0.0f);

    glColor3f(1.0f, 0.0f, 1.0f);
    glVertex3f(0.0f, 0.0f, -60.0f);
    glVertex3f(-60.0f, 0.0f, 0.0f);
    glVertex3f(0.0f, 60.0f, 0.0f);

    glColor3f(1.0f, 0.5f, 0.0f);
    glVertex3f(0.0f, 0.0f, -60.0f);
    glVertex3f(0.0f, -60.0f, 0.0f);
    glVertex3f(-60.0f, 0.0f, 0.0f);

    glColor3f(0.5f, 1.0f, 0.0f);
    glVertex3f(0.0f, 0.0f, -60.0f);
    glVertex3f(60.0f, 0.0f, 0.0f);
    glVertex3f(0.0f, -60.0f, 0.0f);

    glEnd();
    glPopMatrix();
    glutSwapBuffers();
}

void SetupRC(void)
{
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glShadeModel(GL_FLAT);

    // проверка глубины не позволяет дальним граням перекрывать ближние из-за
    // порядка рисования
    glEnable(GL_DEPTH_TEST);
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;
    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;
    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;
    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    if (xRot > 355.0f)
        xRot = 0.0f;
    if (xRot < 0.0f)
        xRot = 355.0f;
    if (yRot > 355.0f)
        yRot = 0.0f;

```

```

        if (yRot < 0.0f)
            yRot = 355.0f;

        glutPostRedisplay();
    }

void ChangeSize(int w, int h)
{
    GLfloat nRange = 100.0f;

    if (h == 0)
        h = 1;

    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // расширение более длинной оси наблюдаемого объема сохраняет пропорции
    // октаэдра
    if (w <= h)
        glOrtho(-nRange, nRange, -nRange * h / w, nRange * h / w,
                -nRange, nRange);
    else
        glOrtho(-nRange * w / h, nRange * w / h, -nRange, nRange,
                -nRange, nRange);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

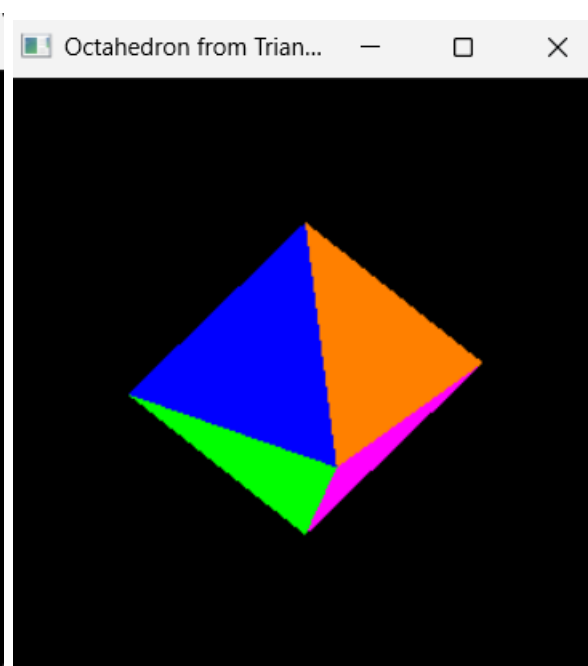
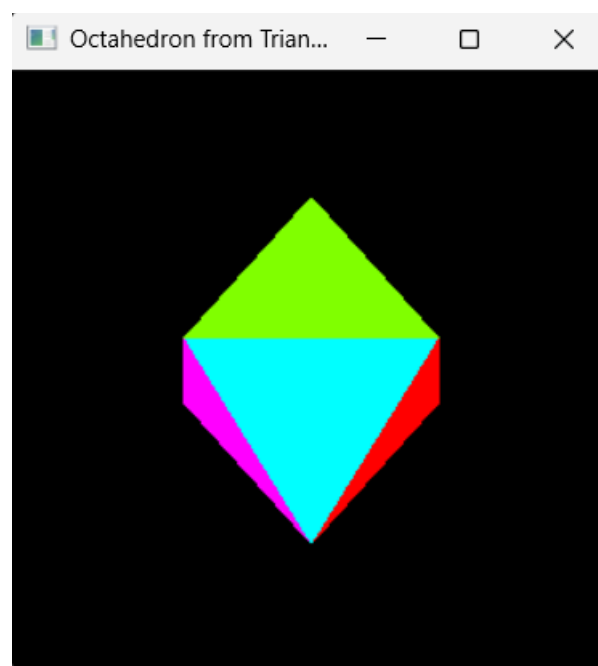
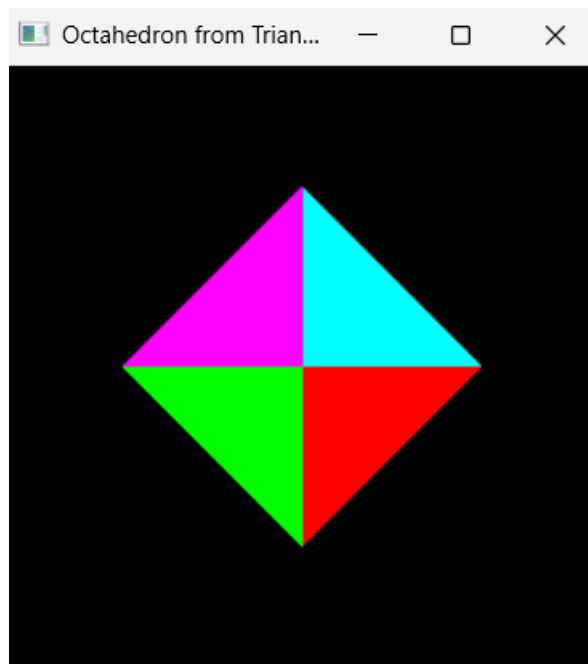
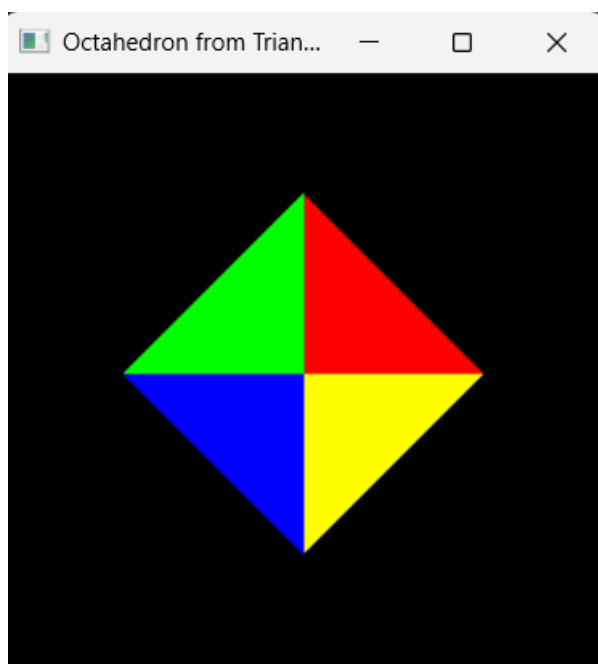
int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutCreateWindow("Octahedron from Triangles");
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);
    SetupRC();
    glutMainLoop();

    return 0;
}

```

В программе с помощью `GL_TRIANGLES` построен октаэдр из восьми треугольных граней. Для граней соблюден согласованный внешний порядок обхода вершин, а проверка глубины обеспечивает правильное перекрытие поверхностей при вращении.

Результаты работы программы 6



Задание:

- 8) Для [Листинга 7](#) продемонстрировать работу директив [GL_TRIANGLE_STRIP](#) и [GL_TRIANGLE_FAN](#).

Листинг 7

```
#include "glut.h"
#include <math.h>

#define GL_PI 3.1415f

static GLfloat xRot = 0.0f;
static GLfloat yRot = 0.0f;

void RenderScene(void)
{
    GLfloat angle;

    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glPushMatrix();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);

    // после первых двух вершин каждая новая вершина GL_TRIANGLE_STRIP создает
    // следующий треугольник
    glBegin(GL_TRIANGLE_STRIP);
    for (int i = 0; i < 5; ++i)
    {
        GLfloat x = -90.0f + i * 20.0f;
        GLfloat z = -20.0f + i * 10.0f;

        // попарное задание нижней и верхней вершин образует непрерывную ленту
        // без разрывов
        glColor3f(1.0f, 0.0f, 0.0f);
        glVertex3f(x, -30.0f, z);
        glColor3f(0.0f, 1.0f, 0.0f);
        glVertex3f(x, 30.0f, z);
    }
    glEnd();

    // первая вершина GL_TRIANGLE_FAN становится общей вершиной всех
    // треугольников веера
    glBegin(GL_TRIANGLE_FAN);
    glColor3f(1.0f, 1.0f, 1.0f);
    glVertex3f(45.0f, 0.0f, 50.0f);

    for (int i = 0; i <= 12; ++i)
    {
        angle = 2.0f * GL_PI * i / 12.0f;
```

```

        if (i % 2 == 0)
            glColor3f(0.0f, 0.0f, 1.0f);
        else
            glColor3f(1.0f, 1.0f, 0.0f);
        glVertex3f(45.0f + 30.0f * cos(angle),
                  30.0f * sin(angle), -30.0f);
    }
    // повторение первой точки окружности в конце цикла замыкает боковую
    поверхность конуса
    glEnd();

    glPopMatrix();
    glutSwapBuffers();
}

void SetupRC(void)
{
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glShadeModel(GL_FLAT);

    // проверка глубины оставляет видимыми только ближайшие части треугольников
    glEnable(GL_DEPTH_TEST);
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;
    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;
    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;
    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    if (xRot > 355.0f)
        xRot = 0.0f;
    if (xRot < 0.0f)
        xRot = 355.0f;
    if (yRot > 355.0f)
        yRot = 0.0f;
    if (yRot < 0.0f)
        yRot = 355.0f;

    glutPostRedisplay();
}

void ChangeSize(int w, int h)
{
    GLfloat nRange = 120.0f;

```

```

    if (h == 0)
        h = 1;

    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    if (w <= h)
        glOrtho(-nRange, nRange, -nRange * h / w, nRange * h / w,
                -nRange, nRange);
    else
        glOrtho(-nRange * w / h, nRange * w / h, -nRange, nRange,
                -nRange, nRange);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

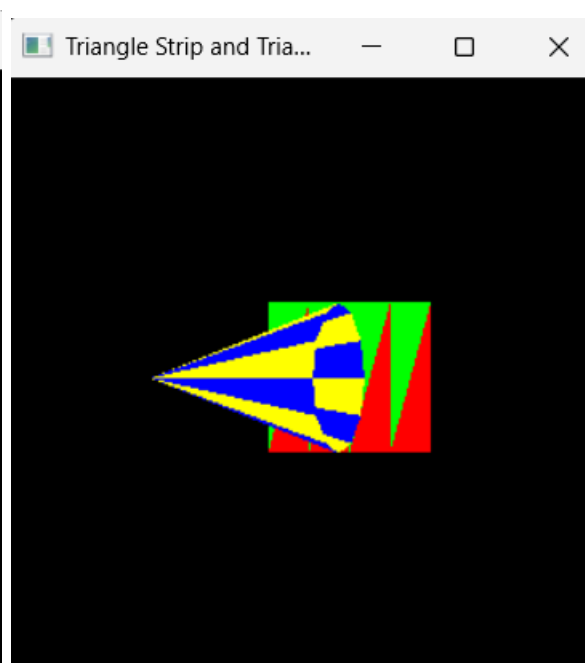
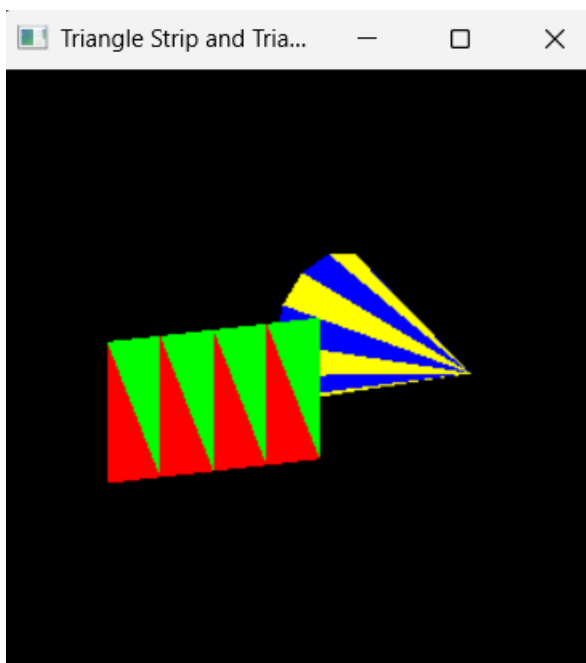
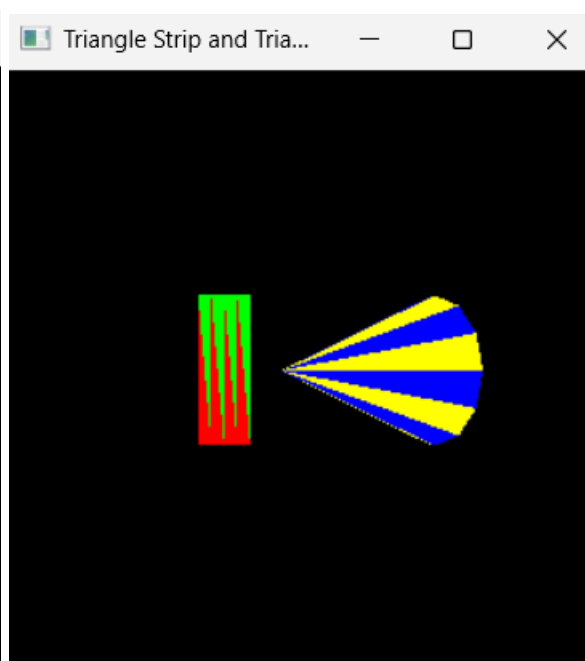
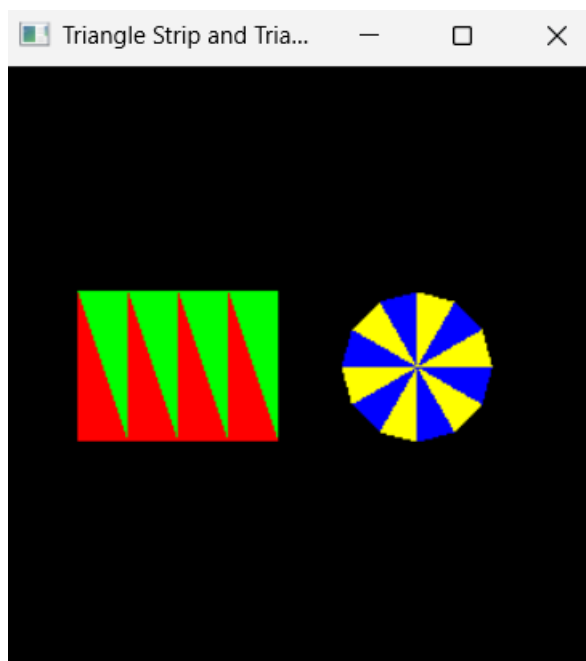
int main(int argc, char* argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutCreateWindow("Triangle Strip and Triangle Fan");
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);
    SetupRC();
    glutMainLoop();

    return 0;
}

```

В программе продемонстрированы два способа объединения треугольников. GL_TRIANGLE_STRIP используется для построения связанной треугольной ленты, а GL_TRIANGLE_FAN — для построения боковой поверхности конуса.

Результаты работы программы 7



Задание:

- 9) Для [Листинга 8](#) для произвольно заданной фигуры составленной из не менее чем 6 треугольников продемонстрировать работу функции `glShadeModel`.

Листинг 8

```
#include "glut.h"
#include <math.h>

#define GL_PI 3.1415f

static GLfloat xRot = 0.0f;
static GLfloat yRot = 0.0f;
static GLenum shadeModel = GL_FLAT;

void RenderScene(void)
{
    GLfloat angle;

    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    // режим затенения задается до glBegin и действует на все последующие
    // многоугольники
    glShadeModel(shadeModel);

    glPushMatrix();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);

    // первая вершина веера является общей вершиной всех восьми треугольников
    // конуса
    glBegin(GL_TRIANGLE_FAN);
    glColor3f(1.0f, 1.0f, 1.0f);
    glVertex3f(0.0f, 0.0f, 70.0f);

    for (int i = 0; i <= 8; ++i)
    {
        angle = 2.0f * GL_PI * i / 8.0f;

        // условие i <= 8 повторяет начальную точку окружности при последней
        // итерации
        // цвет задается вершине, а способ его распространения по грани
        // определяет glShadeModel
        if (i % 3 == 0)
            glColor3f(1.0f, 0.0f, 0.0f);
        else if (i % 3 == 1)
            glColor3f(0.0f, 1.0f, 0.0f);
        else
            glColor3f(0.0f, 0.0f, 1.0f);
```

```

        glVertex3f(55.0f * cos(angle), 55.0f * sin(angle), -50.0f);
    }
    // повторенная точка окружности замыкает веер, поэтому девять точек контура
    // дают восемь граней
    glEnd();

    glPopMatrix();
    glutSwapBuffers();
}

void SetupRC(void)
{
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);

    // проверка глубины обеспечивает правильное перекрытие граней при вращении
    // конуса
    glEnable(GL_DEPTH_TEST);
}

void ProcessMenu(int value)
{
    // режим GL_FLAT использует цвет последней вершины грани, а GL_SMOOTH
    // интерполирует цвета вершин
    if (value == 1)
        shadeModel = GL_FLAT;
    else
        shadeModel = GL_SMOOTH;

    // выбранный режим сохраняется в shadeModel и применяется при следующей
    // перерисовке
    glutPostRedisplay();
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;
    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;
    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;
    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    if (xRot > 355.0f)
        xRot = 0.0f;
    if (xRot < 0.0f)
        xRot = 355.0f;
    if (yRot > 355.0f)
        yRot = 0.0f;
    if (yRot < 0.0f)

```

```

        yRot = 355.0f;

        glutPostRedisplay();
    }

    void ChangeSize(int w, int h)
    {
        GLfloat nRange = 100.0f;

        if (h == 0)
            h = 1;

        glViewport(0, 0, w, h);
        glMatrixMode(GL_PROJECTION);
        glLoadIdentity();

        if (w <= h)
            glOrtho(-nRange, nRange, -nRange * h / w, nRange * h / w,
                    -nRange, nRange);
        else
            glOrtho(-nRange * w / h, nRange * w / h, -nRange, nRange,
                    -nRange, nRange);

        glMatrixMode(GL_MODELVIEW);
        glLoadIdentity();
    }

    int main(int argc, char* argv[])
    {
        glutInit(&argc, argv);
        glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
        glutCreateWindow("Flat and Smooth Shading");
        glutReshapeFunc(ChangeSize);
        glutSpecialFunc(SpecialKeys);
        glutDisplayFunc(RenderScene);
        SetupRC();

        glutCreateMenu(ProcessMenu);
        glutAddMenuEntry("GL_FLAT", 1);
        glutAddMenuEntry("GL_SMOOTH", 2);
        glutAttachMenu(GLUT_RIGHT_BUTTON);

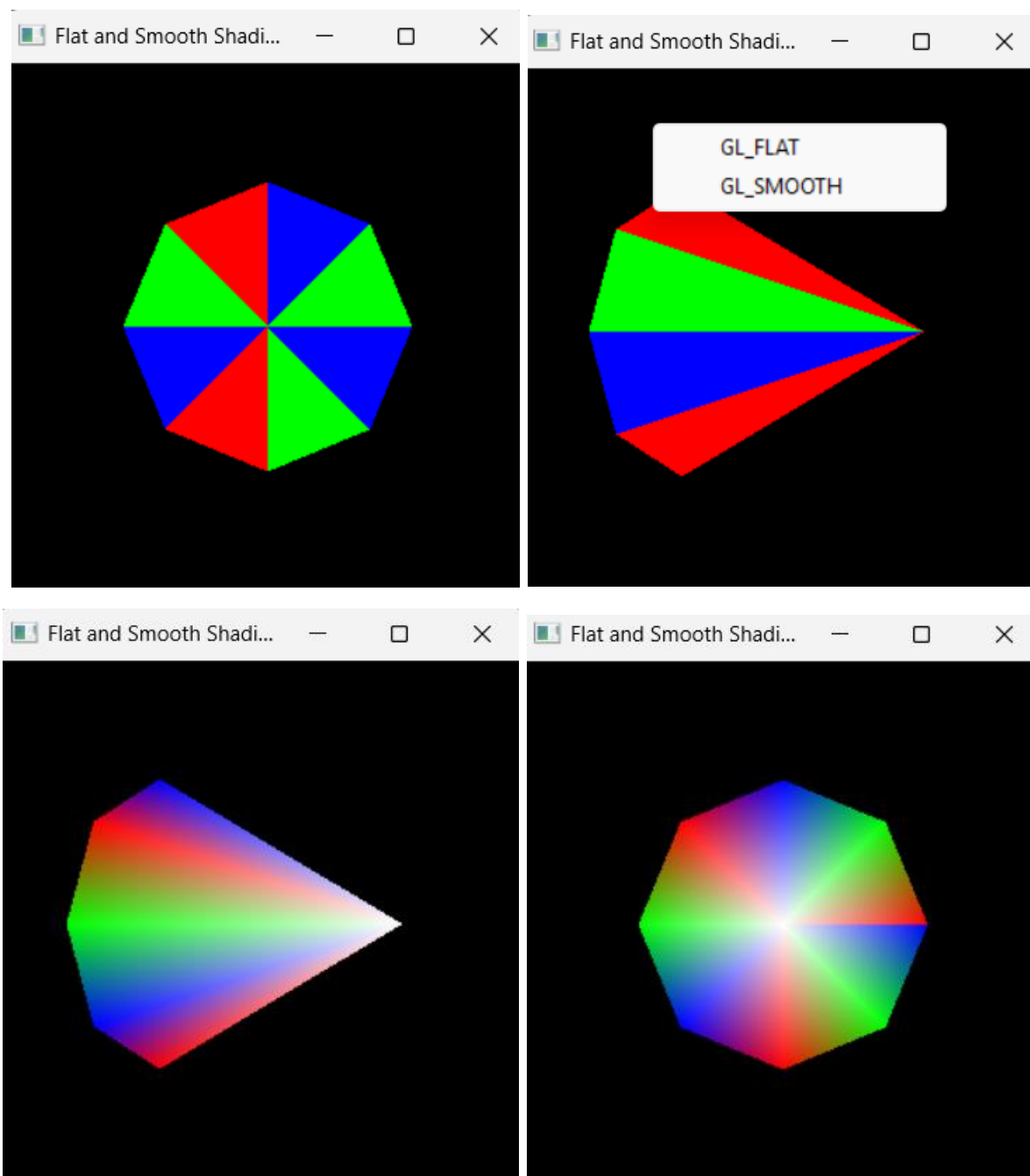
        glutMainLoop();

        return 0;
    }

```

В программе построен конус из восьми треугольных граней и исследована работа `glShadeModel`. В режиме `GL_FLAT` каждая грань имеет сплошной цвет, а в режиме `GL_SMOOTH` выполняется плавная интерполяция цветов между вершинами.

Результаты работы программы 8



Задание:

- 10) Для [Листинга 9](#) на наглядном примере продемонстрировать работу проверки глубины для технологии удаления скрытых поверхностей. На наглядном примере продемонстрировать работу технологии отбора задних граней.

Листинг 9

```
#include "glut.h"
#include <math.h>

#define GL_PI 3.1415f

static GLfloat xRot = 0.0f;
static GLfloat yRot = 0.0f;
static bool bDepth = false;
static bool bCull = false;

void RenderScene(void)
{
    GLfloat angle;

    // буфер глубины необходимо очищать перед построением каждого нового кадра
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // отбор прекращает обработку граней, которые определены как обратные
    if (bCull)
        glEnable(GL_CULL_FACE);
    else
        glDisable(GL_CULL_FACE);

    // проверка глубины оставляет в каждом пикселе фрагмент ближайшей грани
    if (bDepth)
        glEnable(GL_DEPTH_TEST);
    else
        glDisable(GL_DEPTH_TEST);

    glPushMatrix();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);

    // первая вершина веера является общей вершиной всех боковых граней конуса
    glBegin(GL_TRIANGLE_FAN);
    glColor3f(1.0f, 1.0f, 1.0f);
    glVertex3f(0.0f, 0.0f, 70.0f);

    for (int i = 0; i <= 16; ++i)
    {
        angle = 2.0f * GL_PI * i / 16.0f;
```

```

    // при GL_FLAT каждая грань получает цвет последней заданной вершины
    if (i % 2 == 0)
        glColor3f(1.0f, 0.0f, 0.0f);
    else
        glColor3f(0.0f, 1.0f, 0.0f);

    glVertex3f(50.0f * sin(angle), 50.0f * cos(angle), -40.0f);
}
// повторение первой точки окружности замыкает боковую поверхность
glEnd();

// основание строится отдельным веером с центром в качестве общей вершины
glBegin(GL_TRIANGLE_FAN);
glColor3f(0.0f, 0.0f, 1.0f);
glVertex3f(0.0f, 0.0f, -40.0f);

for (int i = 0; i <= 16; ++i)
{
    // обратный проход окружности направляет лицевую сторону основания наружу
    angle = 2.0f * GL_PI * (16 - i) / 16.0f;
    glVertex3f(50.0f * sin(angle), 50.0f * cos(angle), -40.0f);
}
glEnd();

glPopMatrix();
glutSwapBuffers();
}

void SetupRC(void)
{
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glShadeModel(GL_FLAT);

    // по аналогии с примером методички лицевыми считаются грани с обходом по
    // часовой стрелке
    glFrontFace(GL_CW);
}

void ProcessMenu(int value)
{
    // пункты меню изменяют только состояния проверки глубины и отбора граней
    if (value == 1)
        bDepth = true;
    else if (value == 2)
        bDepth = false;
    else if (value == 3)
        bCull = true;
    else if (value == 4)
        bCull = false;
}

```

```

        glutPostRedisplay();
    }

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;
    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;
    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;
    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    if (xRot > 355.0f)
        xRot = 0.0f;
    if (xRot < 0.0f)
        xRot = 355.0f;
    if (yRot > 355.0f)
        yRot = 0.0f;
    if (yRot < 0.0f)
        yRot = 355.0f;

    glutPostRedisplay();
}

void ChangeSize(int w, int h)
{
    GLfloat nRange = 100.0f;

    if (h == 0)
        h = 1;

    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // изменение границ glOrtho компенсирует пропорции окна без искажения конуса
    if (w <= h)
        glOrtho(-nRange, nRange, -nRange * h / w, nRange * h / w,
                -nRange, nRange);
    else
        glOrtho(-nRange * w / h, nRange * w / h, -nRange, nRange,
                -nRange, nRange);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

int main(int argc, char* argv[])
{

```

```

glutInit(&argc, argv);
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
glutCreateWindow("Depth Test and Back-Face Culling");
glutReshapeFunc(ChangeSize);
glutSpecialFunc(SpecialKeys);
glutDisplayFunc(RenderScene);
SetupRC();

glutCreateMenu(ProcessMenu);
glutAddMenuEntry("Depth test ON", 1);
glutAddMenuEntry("Depth test OFF", 2);
glutAddMenuEntry("Back-face culling ON", 3);
glutAddMenuEntry("Back-face culling OFF", 4);
glutAttachMenu(GLUT_RIGHT_BUTTON);

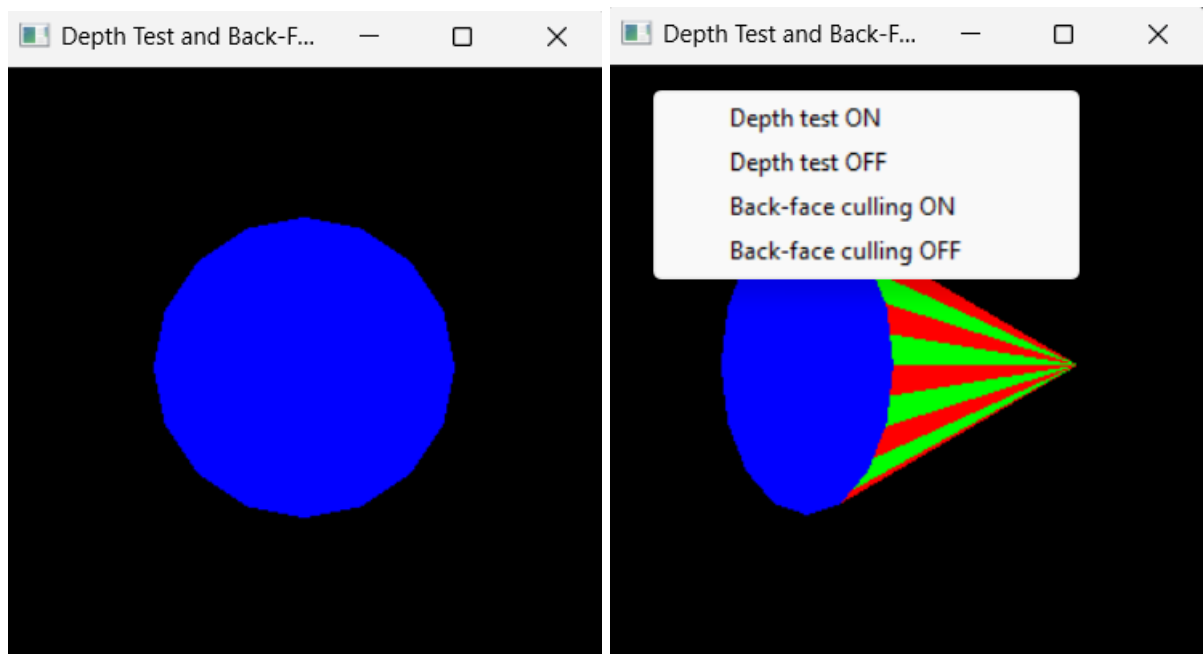
glutMainLoop();

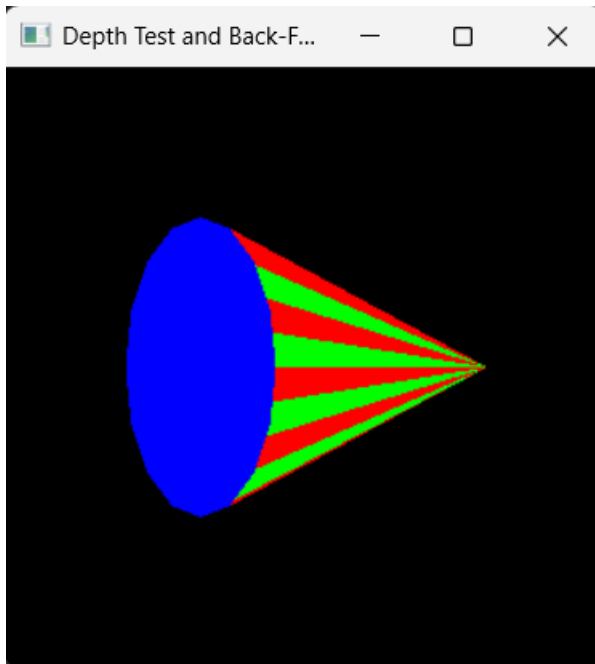
return 0;
}

```

В программе на примере замкнутого конуса исследованы проверка глубины и отбор задних граней. Через контекстное меню режимы GL_DEPTH_TEST и GL_CULL_FACE включаются и выключаются независимо, что позволяет сравнить результаты их работы.

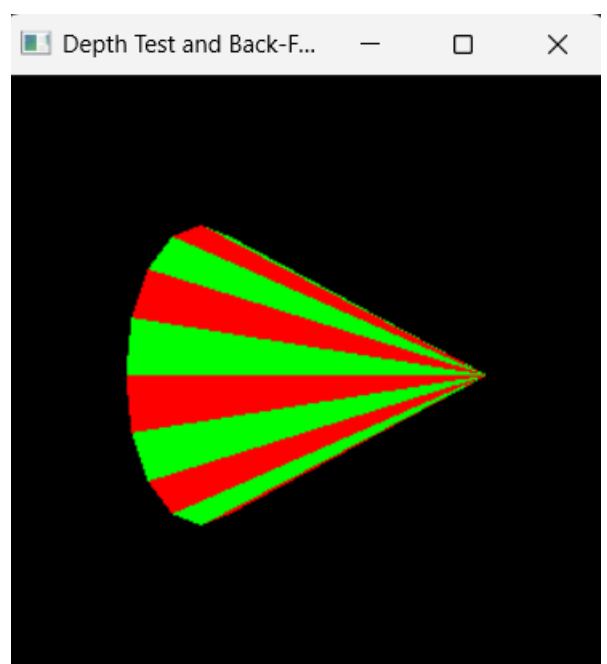
Результаты работы программы 9





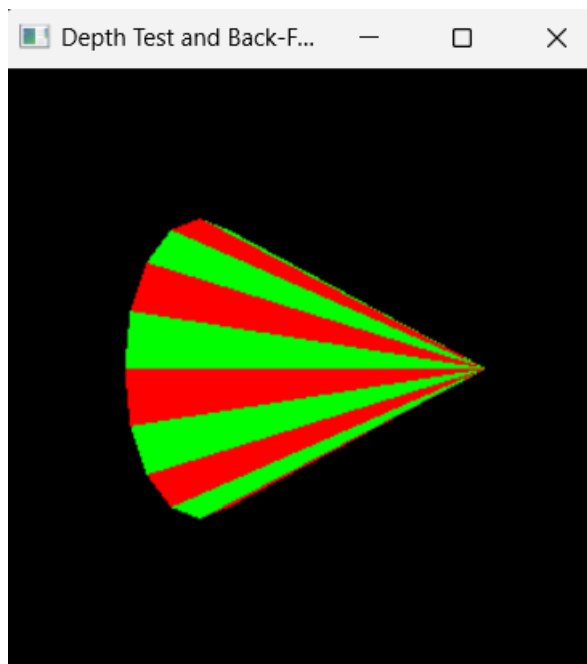
Depth test OFF + Back-face culling OFF

(Проверка глубины и отбор задних граней выключены)



Depth test ON + Back-face culling OFF

(Проверка глубины включена)



Depth test OFF + Back-face culling ON

(Отбор задних граней включён при выключенной проверке глубины)

Вывод: в ходе работы были получены практические навыки по работе с графическими примитивами OpenGL, а также по применению к ним эффектов средствами машины состояний и использования проверки глубины.